NAME:DARSHAN D

REG NO:192524117

6.Write the python program for Cript-Arithmetic problem

Program:

```
from itertools import permutations


for p in permutations(range(10), 8):
    s,e,n,d,m,o,r,y = p
    if s == 0 or m == 0:
        continue
    send = 1000*s+100*e+10*n+d
    more = 1000*m+100*o+10*r+e
    money = 10000*m+1000*o+100*n+10*e+y
    if send+more == money:
        print("SEND =", send)
        print("MORE =", more)
        print("MONEY =", money)
        break
```

Output:

```
>>>
============ RESTART:
SEND = 9567
MORE = 1085
MONEY = 10652
>>>
```

7. Write the python program for Vacuum Cleaner problem

Program:

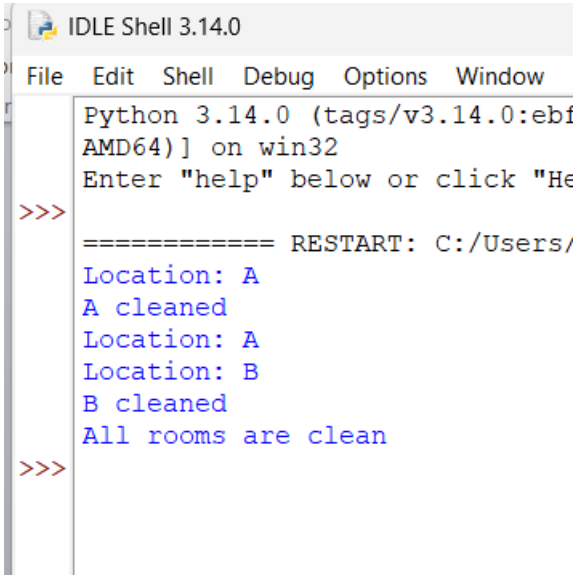
```
room = {'A': 'Dirty', 'B': 'Dirty'}
```

```
location = 'A'


while 'Dirty' in room.values():
    print("Location:", location)
    if room[location] == 'Dirty':
        room[location] = 'Clean'
        print(location, "cleaned")
    else:
        location = 'B' if location == 'A' else 'A'


print("All rooms are clean")
```

Output:



8. Write the python program to implement BFS.

Program:

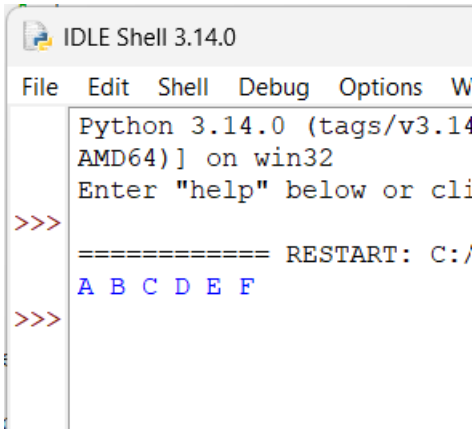
```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
```

```
    'D': [],
    'E': [],
    'F': []
}

visited = []
queue = ['A']

while queue:
    node = queue.pop(0)
    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        queue.extend(graph[node])
```

Output:



9. Write the python program to implement DFS.

Program:

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
```

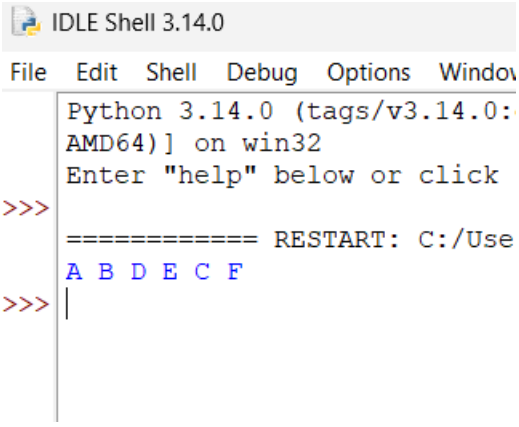
```
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

visited = set()

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for i in graph[node]:
            dfs(i)

dfs('A')
```

Output:



10. Write the python program to implement A* algorithm

Program:

```
graph = {
```

```
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 1), ('E', 5)],
    'C': [('F', 2)],
    'D': [], 'E': [], 'F': []
}

h = {'A': 4, 'B': 3, 'C': 2, 'D': 0, 'E': 1, 'F': 0}

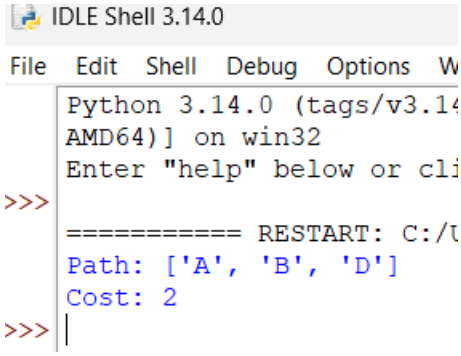
open_list = [(h['A'], 0, 'A', ['A'])]

while open_list:
    open_list.sort()
    f, g, node, path = open_list.pop(0)

    if node == 'D':
        print("Path:", path)
        print("Cost:", g)
        break

    for next_node, cost in graph[node]:
        new_g = g + cost
        open_list.append((new_g + h[next_node], new_g, next_node, path +
[next_node]))
```

Output:


IDLE Shell 3.14.0
File Edit Shell Debug Options W
Python 3.14.0 (tags/v3.14
AMD64)] on win32
Enter "help" below or cli
>>>
========== RESTART: C:/U
Path: ['A', 'B', 'D']
Cost: 2
>>>